

SYSTEM ARCHITECTURE & BENCHMARK SUITE

Architecture: Sensor-Bound Gated Detection of Injected and Synthetic Video

Status: research prototype, October 2026. All numbers in this document come from `benchmarks/run_benchmark.py` ; the raw output is in `benchmarks/results/` .

1. Summary

Remote identity verification assumes the video comes from a real camera pointed at a real person. Two attack families break that assumption:

- **Injection attacks** bypass the lens entirely. Virtual cameras (OBS, v4l2loopback), hooked capture APIs or emulators feed the application pre-rendered or real-time-generated frames. Presentation-attack detection (ISO/IEC 30107-3) never sees a physical artefact to detect.
- **Synthetic content** is the payload itself: face swaps, reenactment, talking heads and text-to-video. It is often delivered through an injection channel.

This project builds a **gated pipeline** that runs cheap, decisive checks first and spends pixel analysis only on sessions that survive them:

Gate	Question	Cost (measured, laptop CPU)
1a Passive attestation	Is the capture node a physical camera in an untampered environment?	~10–25 ms
1b Active sensor challenge	Do the frames respond to a random command sent to that physical sensor?	~1.6 s capture time, <5 ms compute
2 Sensor-bound PRNU	Do the pixels carry the fingerprint of <i>that</i> attested sensor?	~0.2 s / 45 frames at 640×480
3 Temporal / spectral	Is the facial content temporally consistent?	~0.2–0.6 s / 45–90 frames at 640×480

The contribution is the binding between the gates, not any single detector.

- Gate 1 produces a hardware identity: USB VID:PID + serial/port, and the V4L2 node.
- Gate 1b proves the *open stream* is driven by that sensor right now.
- Gate 2 proves the *pixels* were formed on that sensor's silicon, using a fingerprint enrolled under the same identity.

Each check covers a documented blind spot of the others (Section 5). None of it uses a trained model, so it is cheap, explainable, and does not degrade when a new generator is released.

2. Threat model

The defender controls the verification software on the user's machine (the KYC client). The attacker controls the frames presented to it. They may have user-level control of the machine and can run any software: virtual cameras, deepfake tools, screen recorders.

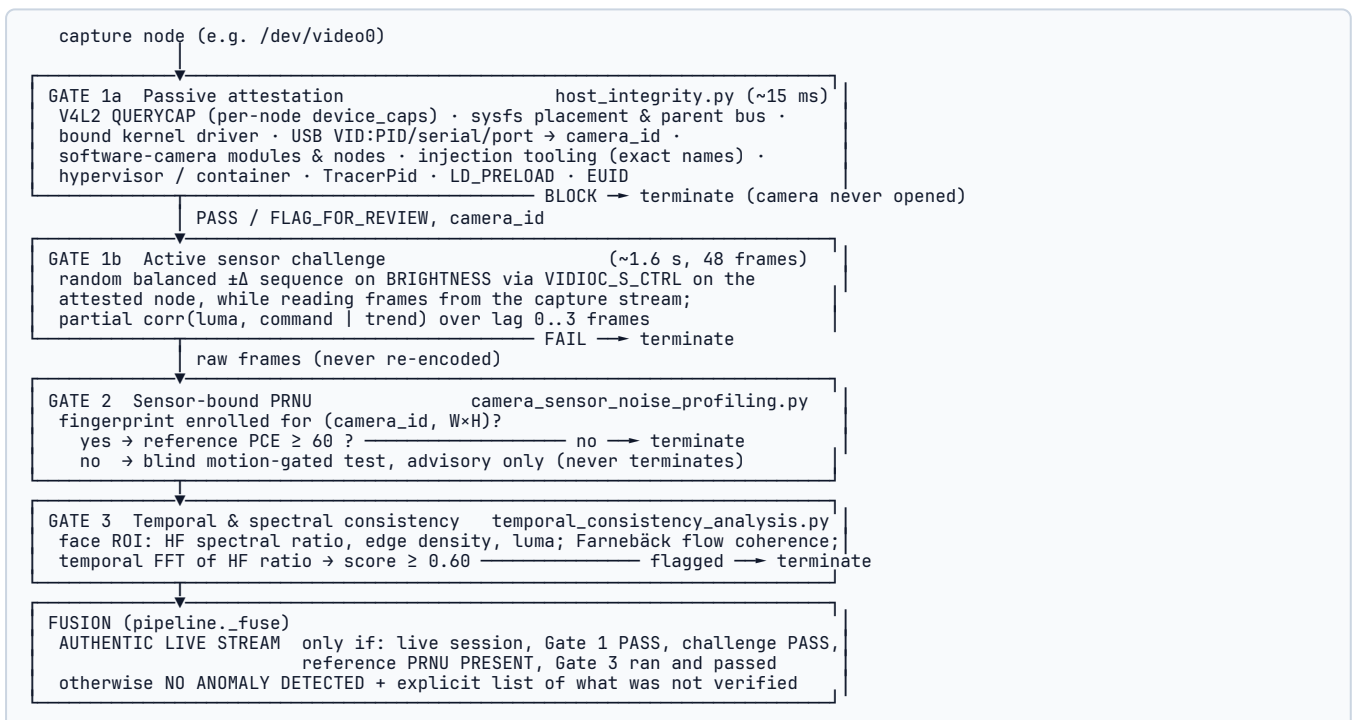
ID	Attack	Example tooling
A1	Virtual camera device feeding pre-recorded or synthetic video	OBS Virtual Camera → v4l2loopback, akvcam, ManyCam
A2	Real-time face swap on the real webcam, output through a virtual camera	DeepFaceLive → OBS → v4l2loopback
A3	Hooked capture API inside the verifying process (frames substituted after the driver)	LD_PRELOAD shim, Frida, ptrace
A4	Emulator / VM / container with a virtual camera	QEMU, VirtualBox, Android AVD, Docker
A5	Fully synthetic video (text/image-to-video, talking head)	Veo 3, Kling, HunyuanVideo, Grok, HeyGen, SadTalker
A6	Photo re-animation (one real photo, animated)	FOMM, MRAA, LivePortrait-style tools
A7	Replay of genuine footage of the victim, recorded earlier on <i>another</i> camera	Phone recording replayed via A1
A8	Replay of genuine footage recorded earlier on <i>this same</i> camera	Recording of a previous session replayed via A1/A3

Out of scope:

- kernel-level compromise (a malicious camera driver), hardware man-in-the-middle on the USB bus
- physical presentation attacks (masks, screens held up to the lens), which are ISO 30107-3 PAD territory

Also out of scope here is an adaptive attacker who reads the physical sensor's control state and re-renders synthetic frames to match it in real time (see Section 8).

3. Architecture



Enrollment (`pipeline.enroll_live_camera`) runs Gate 1a and 1b, captures 150 frames and stores $K = \sum W \cdot I / \sum I^2$ under `fingerprints/<camera_id>_<W>x<H>.npy` . A software camera cannot be enrolled, because enrollment refuses unless both checks pass.

Files (uploads) have no live device. Gate 1 returns `UNATTESTED_ORIGIN` with container metadata for information only. Gate 2 is enforced only if a reference fingerprint is supplied. A file can never receive the `AUTHENTIC LIVE STREAM` verdict.

4. Layers in detail

4.1 Gate 1a: passive attestation (`host_integrity.probe_host_integrity`)

A node is a **physical camera** only if all of the following hold:

1. **It is a capture node.** `VIDIOC_QUERYCAP` reports `V4L2_CAP_VIDEO_CAPTURE` or `_MPLANE` in `device_caps` (the per-node field, used when `V4L2_CAP_DEVICE_CAPS` is set). A UVC camera exposes a second node for metadata (`/dev/video1` on the test laptop, `device_caps = 0x04a00000`), and it is not a camera.
2. **It is not virtual.** The node's sysfs entry is not under `/sys/devices/virtual/` and it has a `device` parent.
3. **It hangs off real hardware.** The parent is on a USB or PCI bus, and no layer (`QUERYCAP` driver, bound kernel driver, `bus_info`) names a known software-camera driver (`v4l2_loopback` , `akvcam` , `vivid` , `vimc` , `vcam`). A node that accepts frames from user space (`VIDEO_OUTPUT` + `VIDEO_CAPTURE`) is treated as virtual too.

The card label is *not* trusted: `v4l2loopback` lets the attacker set it to "Integrated Camera". The unit tests build a fake sysfs tree with exactly that spoof.

Camera identity: the sysfs walk-up to the USB device yields `idVendor:idProduct` , `serial` and the port path, e.g. `usb-5986-216a-1-3` . This identity keys the Gate 2 fingerprint.

Host context:

- loaded software-camera modules (`/proc/modules`) and other virtual nodes
- injection tooling matched on **exact** process names (`obs` , `droidcam` , `manycam` , `deepfacelive` ...) or command-line patterns (`-f v4l2 ... /dev/videoN` , `v4l2sink` , `pyvirtualcam`)

These only raise `FLAG_FOR_REVIEW` when the selected camera is physical.

Environment:

- **hypervisor:** CPUID hypervisor bit, DMI vendor strings (word-boundary matched), `/sys/hypervisor/type`
- **container:** `/.dockerenv` , `/run/.containerenv` , PID 1 cgroup
- **hooks:** `TracerPid != 0` , `LD_PRELOAD` or `/etc/ld.so.preload`
- **privilege:** EUID 0

Any of these BLOCKs.

Bugs fixed relative to the merged branches (each caused wrong verdicts on the test laptop):

Bug	Effect
Substring process matching (<code>"obs"</code> in comm)	<code>obsidian</code> (a note-taking app) made the real webcam "virtual" → <code>FLAG_FOR_REVIEW</code>
Used device-wide <code>capabilities</code> , not per-node <code>device_caps</code>	the UVC metadata node was listed as a selectable camera
Virtual-camera finding was <code>FLAG_FOR_REVIEW</code> with <code>passed=True</code>	an injected stream from a loopback device proceeded through every gate
<code>/usr/bin/su</code> treated as a root indicator	true on every Linux desktop; shown as a compromise signal
UI fell back to hard-coded <code>LENOVO / LOQ 15ARP9 / AMD Ryzen</code> strings	fabricated telemetry whenever a field was missing

4.2 Gate 1b: active sensor challenge (`host_integrity.run_sensor_challenge`)

Passive checks trust what the OS reports. The challenge tests causality instead: *if I change the physical sensor, do these frames change?*

1. Pick a writable integer image control on the attested node. Brightness comes first, then gamma, then gain. Ranges come from `VIDIOC_QUERYCTRL` .
2. Draw a random, balanced sequence of 12 slots ($\pm\Delta$, $\Delta = 10\%$ of the control range, at least 5 sign changes) from `random.SystemRandom` .
3. For each slot, set the control via `VIDIOC_S_CTRL` on a second fd to the attested node, and read 4 frames from the *capture stream under test*. For each frame, record the **median per-pixel brightness change** against the first frame

of the test (80×60 thumbnail). The brightness command shifts every pixel equally, while a person moving changes only part of the image, so the median stays on the commanded shift as long as less than half the frame moves. (The first version used the mean luma of the centre; one genuine trial failed at $r = 0.25$ while someone moved.) Each frame is labelled with the command that was active **when the sensor captured it**, using the driver's CLOCK_MONOTONIC buffer timestamp (`CAP_PROP_POS_MSEC`). At low frame rates (the webcam drops to 9–10 fps in dim light) frames wait in the driver queue for a variable time, which made the delay jitter between 1 and 2 frames and pushed genuine r down to 0.74–0.79. Timestamp labelling removes this: at 10 fps all checks scored $r \geq 0.99$ with zero lag.

4. Restore the original value in a `finally` block. Verified after every run: brightness returned to 128.
5. Statistic: the **partial correlation** of luma with the command, controlling for a linear trend (auto-exposure drift), maximised over 0–3 frames of latency. Both series are residualised on `[1, t]`. Detrending only the luma (the first version) lets the trend fit absorb part of the square wave and pushed genuine r down to 0.82.
6. PASS if $r \geq 0.9$ and the $\pm$ effect is ≥ 6 grey levels. FAIL needs both of 2 attempts to fail. If no writable control exists, the result is UNSUPPORTED (recorded as a limitation, not a block).

On the test webcam the response is a clean square wave with exactly one frame of latency ($\Delta = \pm 25$ brightness units → about 52 grey levels). Live, false-pass and replay numbers are in Section 7.

Frames are only ever read through the capture stream, and the stimulus only ever goes to the physical node. So a loopback device (A1/A2), a hooked capture call (A3), or a replay of anything (A7/A8) does not respond. A replay cannot anticipate a sequence drawn after the session starts.

4.3 Gate 2: sensor-bound PRNU (`camera_sensor_noise_profiling.py`)

Model. $I = I_0(1 + K) + \theta$. K is the PRNU gain pattern: pixel-locked, sensor-unique, and about 1% in amplitude (Lukáš et al. 2006).

Residual. $W = (I - \mu_3) \cdot \min(1, \sigma^2/v)$.

- v is the minimum local variance over 3/5/7/9 windows and $\sigma^2 = 3$. This is a spatial analogue of Mihçak's denoiser, so texture is attenuated and flat regions pass.
- Saturated pixels are zeroed.

Fingerprint. $K = \sum W \cdot I / \sum I^2$ (MLE; Chen et al. 2008). Then:

- rows and columns are zero-meant
- in the DFT domain, every magnitude above 3× its local median is clamped. This removes the JPEG/H.264 block grid and CFA periodicities shared by all cameras and codecs.

Reference mode (enforced). PCE (Goljan et al. 2009) between the cleaned sum of residuals and $\sum I \cdot K_{\text{ref}}$, taken at the strongest peak within **± 2 px of zero shift**. The match threshold is 60. The search is needed because genuine sessions of the same webcam were measured with the peak at (0, ± 2) in 27 of 56 session pairs, and at (−1, 0) for a live enrollment. The camera's scaler/crop phase shifts slightly between stream starts. Scoring only (0, 0) reads the shoulder of the peak. Measured over the 56 genuine pairs and 64 impostors at 640×480:

Search	genuine min / median	foreign impostor max	animated photo of this camera, max
(0, 0) only	307 / 1,815	13.2	12.5
± 2 px (used)	392 / 2,983	13.5	23.9–25.2
± 3 px	1,102 / 7,480	17.4	36.5–40.8; one live demo run reached 54

± 3 px gives genuine sessions more headroom, but it lets a re-animated photo of this camera's own frame creep towards the threshold (54 of 60 in one live run). ± 2 px keeps every margin wide: genuine $\geq 6.5\times$ above, foreign sources $\geq 4.4\times$ below, own-photo animation $\geq 2.4\times$ below.

A sensor-mode mismatch (different resolution) returns INCONCLUSIVE, because PRNU is only comparable within one readout/scaling mode. A 1280×720 capture down-scaled to 640×480 gave PCE 15 against the 640×480 fingerprint.

Blind mode (advisory).

1. Split the clip into temporally disjoint halves A and B and estimate `K_A`, `K_B`.

2. Compute the **change mask**: pixels whose gain-normalised, low-passed mean image differs by more than 10 grey levels between A and B (then eroded; pixels outside 5–230 are removed, because highlights on the ISP tone-curve shoulder carry little usable PRNU).
3. Compute the masked correlation ρ and its z-score against 48 random circular shifts.
4. Decide: PRESENT if $z \geq 5$ and $\rho \geq 0.05$. ABSENT if $z < 3$ or $\rho < 0.04$. INCONCLUSIVE if less than 4% of the frame changed, or if neither rule applies.

The change mask exists because with a static camera, background *texture* is as pixel-locked as PRNU. **Only a between-half content difference guarantees decorrelation:**

- Motion that oscillates inside each half revisits the same content, so the same texture leaks into both estimates. An earlier variant that also admitted "varies within both halves" was caught by a unit test: it reported PRESENT for rendered frames with periodic motion. It was reverted.
- A static scene is INCONCLUSIVE, never ABSENT. This is the false-positive guard. The 4% minimum came from a measured failure: with 2%, two empty-room sessions whose only motion was a sunlit curtain (about 3% of the frame, mostly moving edges in highlights) were reported ABSENT.

4.4 Gate 3: temporal & spectral consistency (`temporal_consistency_analysis.py` , unchanged)

The module detects the face in each frame with a Haar cascade and measures three things on the face crop: 2D-FFT high-frequency energy ratio, Canny edge density and mean luma. Across frames it computes:

- frame-to-frame instability of those features (flicker)
- Farnebäck optical-flow direction coherence on the face crop
- a temporal FFT of the HF-ratio series (periodic flicker)
- the coefficient of variation of HF energy

These are combined with weights 0.30/0.25/0.25/0.20 and flagged at 0.60. The thresholds are documented in the module as uncalibrated placeholders. Below 8 face frames the module abstains, returning score 0; the fusion layer records that as a limitation instead of a pass.

Pipeline-level changes:

- Gate 3 now receives raw frames. Measured: same verdicts as the old `mp4v` path on all 14 webcam runs.
- The face tracker is reset per clip. It used to carry a stale box from the previous video into the next one.

4.5 Fusion and early termination (`pipeline.py`)

- **Order is by cost.** Gate 1a takes about 15 ms and, if it blocks, the camera is never opened. Gate 1b costs capture time but almost no compute. Gates 2 and 3 cost roughly 0.2–0.6 s of CPU per 45–90 frames at 640×480.
- **Raw frames end to end.** The previous app wrote live frames to `mp4v` and analysed the file, and the file path re-encoded a "fast sample" the same way. That re-encode **erases PRNU**: genuine frames from session C (new pose and light) fell from $PCE \geq 300$ to 14–22 (zero-shift PCE) against their own camera's fingerprint, an automatic false positive. In an unchanged scene, background leakage can mask this (§7.2), which is why the bug looked intermittent. Now frames are decoded once and passed in memory.
- **Verdicts.**
 - Gate 1 BLOCK or challenge FAIL → `DIGITAL INJECTION DETECTED (...)`
 - Reference PRNU mismatch → `DIGITAL INJECTION DETECTED`
 - Gate 3 flag → `DEEPPAKE DETECTED`
 - Otherwise, `AUTHENTIC LIVE STREAM` only when every live check actually ran and passed. Anything less is `NO ANOMALY DETECTED` with a `limitations` list, e.g. "Gate 3 abstained (face found in 0 frames)" or "no enrolled fingerprint: PRNU advisory".

4.6 Live operation (`live_engine.py` , `app_pages/live.py`)

The app runs the gates **continuously** on an open camera instead of on a recorded clip:

- A **capture thread** owns the camera and streams frames into a ring buffer. It runs the Gate 1b challenge at start, every 10 s, and immediately whenever the fingerprint score drops below the threshold (an adaptive re-check). Each check takes ~1.6 s and visibly flashes the picture.
- An **analysis thread** re-runs Gate 2 (reference PCE) and Gate 3 (temporal score) once per second on the last 45 non-challenge frames. Every threshold crossing is logged as an event ("sensor fingerprint lost", "liveness check

failed", "motion anomaly"), and the page shows the live verdict, per-gate tiles and the 60-second history of each score.

- A shared **compute lock** pauses analysis while a challenge runs. Without it, CPU contention starved the capture thread, frames queued in the driver, and genuine checks failed ($r \approx 0.75$).
- **Enrollment** happens inside the session: the next 150 non-challenge frames become the fingerprint, and it is refused unless the latest liveness check passed.
- **Camera lifetime:** the camera is released on Stop, when the user leaves the page, when the browser stops polling for 8 s (tab closed), when another session takes the camera over, or at process exit. The page only polls while a session is running. Verified in a browser: released after Stop, and within 11 s of closing the tab. The server binds to `localhost` only.
- **Simulated attacks** for demonstration replace the frames *after* the driver (as a hooked capture call would), while the physical camera keeps receiving the challenge:
 - an AI video makes the fingerprint score collapse within a second ($\approx 5,000 \rightarrow 2$), and the next liveness check fails ($r = 0.27$)
 - a replay of the camera's own last 3 s keeps the fingerprint (PCE $\approx 8,600$) but fails the liveness check ($r = 0.33$)

5. Why the combination matters: coverage

✓ = blocks the attack · ◐ = partial / advisory · ✗ = does not detect. "Measured" marks rows backed by Section 7 data.

Attack	1a passive	1b challenge	2 sensor-bound PRNU	3 temporal	Combined
A1 virtual camera device	✓ (node virtual; unit-tested, see §8)	✓ (measured: replay rejected)	✓ unless the source is this camera (measured: 0/100 foreign sources, max PCE 16.9)	✗	✓
A2 real-time face swap via virtual cam	✓	✓	✗ background keeps real PRNU	✗ (measured: 0/28 DF40 face swaps flagged)	✓
A3 hooked capture inside process	✓ if via <code>LD_PRELOAD</code> /ptrace	✓ (frames ignore stimulus)	✓ if frames are not from this sensor	◐	✓
A4 VM / emulator / container	✓	✓ (no physical control)	✓	◐	✓
A5 fully synthetic video	✗ alone	✓ via channel	✓ (measured: 0/26 T2V crops matched)	✗ (measured: 2/16 flagged)	✓
A6 photo re-animation	✗ alone	✓ via channel	✓ (measured: 0/8; PCE ≤ 25.2 even when the photo came from this camera)	✗ (measured: 0/8 DF40 reenactment)	✓
A7 replay of footage from another camera	✗ alone	✓	✓ (measured: 0/54 crops from 11 other phone cameras)	✗	✓
A8 replay of footage from this camera	✗ alone	✓ (only defence; measured 11/11 replays rejected)	✗ (measured: 24/24 lossy replays matched)	✗	✓ via 1b

Reading the table:

- Gate 3, as currently implemented, contributes almost no measured detection (2/64 fakes, §7.5); the combined column rests on Gates 1a, 1b and 2.
- Gate 2 alone fails on A2 and A8.
- Gate 1a alone fails whenever the attacker gets frames past the OS layer without a visible virtual node.
- Gate 1b alone does not tell you *which* sensor responded.

- Only the combination covers every row.

The binding is also what makes Gate 2 a hard gate at all. A PRNU match is only meaningful against a fingerprint enrolled for the physical device that Gate 1 has attested *for this session*, and enrollment is itself gated by 1a + 1b.

6. Negative results that shaped the design

These are reportable findings in their own right.

1. **Blind PRNU presence does not separate real from fake in the wild.** (Section 7, blind table.)
 - Native phone videos (VISION) mostly show *no* detectable pixel-locked pattern in 90 frames. Electronic stabilisation and ISP denoising are known to break video PRNU (Taspinar et al. 2016; Mandelli et al. 2020).
 - Many text-to-video and face-swap clips *do* show a strong one, with z up to about 200. Generators leave their own fixed spatial fingerprints (Marra et al. 2019), and face swaps keep the source video's background.
 - A blind presence test is therefore never used to block.
2. **The original detector measured scene content, not PRNU.** Real webcam PCE was 7k–63k, the Grok AI video scored 79k, and the toy `fake_sim.mp4` was the only clip it caught. The causes were odd/even frame interleaving and whole-frame correlation of a static scene.
3. **Lossy re-encoding before analysis erases PRNU.** This was the dominant source of false positives in the app. See §4.5.
4. **Scene-content leakage needs a between-half content change.** Within-half variation is not enough (periodic motion). See §4.3.
5. **Challenge statistic.** Detrend both series (partial correlation), or the trend fit eats the signal.
6. **The PRNU peak is not always at zero shift.** Small inter-session alignment offsets (1–2 px) are common on a scaling webcam pipeline. A small shift search is required; this was found through the app's correlation-surface view.

7. Evaluation

7.1 Data

Set	Content	Source
Webcam (genuine)	11 sessions, 120 frames each, one laptop camera (USB 5986:216a). 640×480 MJPEG ×7 and YUYV ×1, 1280×720 ×2, 320×240 ×1. Different times, lighting, subject position.	<code>benchmarks/capture_webcam_sessions.py</code>
VISION (real, other cameras)	11 phone models × {indoor still, indoor move} native, + WhatsApp versions (34 clips, first 90 frames, lossless)	Shullani et al. 2017
DF40 (fake)	48 clips, 12 methods: face-swap (DeepFaceLab, FaceSwap, InSwapper, SimSwap, UniFace, MobileSwap, FaceDancer), reenactment (MRAA, FOMM), talking head (SadTalker, Wav2Lip, HeyGen)	Yan et al. 2024
Text-to-video (fake)	16 clips: Veo 3, Kling, HunyuanVideo (5 each), Grok (1)	HF <code>34data/gen-videos-*</code>
Simulated attacks	photo re-animation and static replay of genuine frames, x264 (CRF 23/32) and <code>mp4v</code> replays of genuine sessions	generated in <code>run_benchmark.py</code>

7.2 Gate 2, enrolled reference (leave-one-session-out, 45 test frames)

Decision threshold $PCE \geq 60$. Every session is enrolled in turn and tested against all the other sessions (45 frames each). Impostors are the central crop of every corpus clip that is at least as large as the sensor mode. Replays and attacks are derived from genuine sessions.

640×480 (8 sessions)	n	PCE min / median / max	accepted
Genuine , other session	56	392 / 2,983 / 21,227	56/56
Impostor: phone native (VISION, 11 cameras)	22	0.3 / 3.3 / 13.5	0/22
Impostor: phone WhatsApp (VISION)	12	1.5 / 4.0 / 7.1	0/12
Impostor: DF40 face-swap	8	0.8 / 1.9 / 10.0	0/8
Impostor: DF40 talking-head	6	0.4 / 1.2 / 4.0	0/6
Impostor: text-to-video	16	1.1 / 2.9 / 10.1	0/16
Attack: photo re-animation of a frame from this camera	7	2.4 / 9.3 / 25.2	0/7
Replay of genuine session, x264 CRF 23	7	920 / 1,495 / 7,202	7/7
Replay of genuine session, x264 CRF 32	7	184 / 456 / 1,824	7/7
Replay of genuine session, OpenCV mp4v	7	1,816 / 2,541 / 12,467	7/7
Static replay of a single genuine frame	7	734 / 1,454 / 9,422	7/7

1280×720 (2 sessions)	n	PCE min / median / max	accepted
Genuine	2	3,418 / 3,785 / 4,153	2/2
Impostors (VISION, DF40, T2V crops)	36	0.2 / 2.7 / 16.9	0/36
Replays: x264 CRF 23 / CRF 32 / mp4v / static frame	1 each	176 / 16 / 385 / 552	3/4
Photo re-animation (this camera)	1	1.0	0/1

Overall: **58/58 genuine accepted, 0/100 foreign impostors accepted**. Lowest genuine PCE is 392 (6.5× the threshold); highest foreign impostor is 16.9 (3.6× below it); a re-animated photo of this camera's own frame reached 25.2. Compute is ~0.2 s per 45-frame verification at 640×480 and ~0.8 s at 1280×720.

Scene-leakage caveat (important). Every genuine session was captured in the same room, with the camera in the same position, so static background texture leaks into the enrolled fingerprint and inflates genuine PCE. The proof is that lossy replays of genuine sessions still match:

- x264 CRF 32 normally destroys webcam PRNU. Session C (different pose and light) dropped to PCE 14–22 after **mp4v**, and to below 0 after CRF 32, against an enrollment from session A (zero-shift PCE).
- Here, with scene and camera pose identical, the same kinds of replay still score PCE 108 to 12,467.

Security implication: leakage only helps streams that already contain this camera's pixels at the same pose, i.e. **A8**, **which Gate 2 cannot stop by design**; Gate 1b covers it. It does not let another camera or a generator through (0/100).

Measurement implication: genuine magnitudes here are optimistic. The multi-device study (§9) must enroll and test in different scenes, and enrollment should use varied content (move the camera, or point it at a plain bright surface).

7.3 Gate 2, blind (advisory)

No decision is taken on this result; it is reported for completeness.

Group	N=45: PRESENT / ABSENT / INCONCL.	N=90: PRESENT / ABSENT / INCONCL.
Real: webcam (11 sessions)	2 / 0 / 9	2 / 0 / 9
Real: phone native (VISION, 22)	1 / 16 / 5	4 / 16 / 2
Real: phone WhatsApp (VISION, 12)	0 / 6 / 6	0 / 9 / 3
Fake: DF40 face-swap (28)	11 / 2 / 15	16 / 4 / 8
Fake: DF40 reenactment (8)	0 / 3 / 5	0 / 3 / 5
Fake: DF40 talking head (12)	2 / 1 / 9	4 / 1 / 7
Fake: text-to-video (16)	3 / 4 / 9	7 / 3 / 6
Attack: photo re-animation (3)	0 / 3 / 0	0 / 1 / 2
Attack: static replay (3)	0 / 0 / 3	0 / 0 / 3

What this shows:

- **On the live webcam the blind test never says ABSENT**, so it causes no false positives; it is INCONCLUSIVE when the scene is static.
- **On in-the-wild video it has no discriminative value.** Real phone videos are mostly ABSENT, and face-swap / text-to-video clips are often PRESENT. This is why it is advisory (§6.1).
- **Re-animation is caught only when it changes enough of the frame.** Periodic animation that revisits the same content falls under the 4% rule and becomes INCONCLUSIVE.

7.4 Gate 1b, active challenge

Measure	Result
Live genuine passes, <code>/dev/video0</code>	20/20 ; partial r 0.91–1.00 (median 1.00), effect 42–59 grey levels, 1.58 s (4.4 s when the retry was needed)
Replayed genuine frames read while the physical sensor was challenged	11/11 rejected
Null false-pass rate: random windows of 109 real luma traces (webcam, VISION, DF40, T2V) scored against random challenge sequences, same median-shift statistic	0 / 32,700 (95% upper bound 9.2×10^{-5} per attempt); null r p99 = 0.59, max = 0.83, threshold 0.90
Low frame rate (1280×720 YUYV, 10 fps), timestamp labelling	5/5, $r \geq 0.99$, lag 0
Live engine (continuous session, check every 10 s)	10/10 consecutive checks passed over two runs, r 0.90–1.00 (one at 0.903, likely motion; the engine retries once before failing)
Control restored after every run	yes (brightness 128 checked with <code>v4l2-ctl</code>)
Gate 1a passive probe latency	median 9.7 ms (9.4–11.9 ms)

The threshold was chosen on this null *before* the final run; the first statistic (luma-only detrending, $r \geq 0.8$, 8 slots) had a 0.27% null false-pass rate.

7.5 Gate 3 (unchanged module), 90 frames

Reported to document the current state; the module was left unchanged by request.

Group	flagged	passed	abstained (< 8 face frames)
Real: webcam (11)	0	2	9
Real: phone native (VISION, 22)	3	11	8
Real: phone WhatsApp (VISION, 12)	0	3	9
Fake: DF40 face-swap (28)	0	28	0
Fake: DF40 reenactment (8)	0	8	0
Fake: DF40 talking head (12)	0	12	0
Fake: text-to-video (16)	2	7	7

Gate 3 currently detects 2 of 64 fakes and flags 3 of 34 genuine phone videos. Its heuristic thresholds are uncalibrated placeholders (as its own docstring says), and the Haar detector misses faces often enough that it abstains on most webcam clips. It is not part of the evidence for the architecture's claim. Fixing it is the main open item (§9, step 5).

7.6 End-to-end live sessions on the test laptop

Session	Verdict	Notes
Not enrolled	NO ANOMALY DETECTED	Gate 1 PASS, challenge $r = 1.00$, PRNU blind INCONCLUSIVE (static scene), Gate 3 abstained (no face)
Enrolled, 90 frames	NO ANOMALY DETECTED	reference PCE = 16,383; only limitation: Gate 3 abstained (no face in view)
Enrolled, 45 frames	NO ANOMALY DETECTED	reference PCE = 14,079
<code>/dev/video1</code> (metadata node)	DIGITAL INJECTION DETECTED (ENVIRONMENT COMPROMISED)	blocked in ~11 ms, camera never opened

Timings for one 90-frame session:

- Gate 1a: 11–17 ms
- challenge: 1.58 s (48 frames at 30 fps)
- Gate 2: 0.42 s
- Gate 3: 0.42–0.64 s
- total: 6.6 s, dominated by frame capture (warm-up + challenge + 3 s of video)

8. Limitations and threats to validity

Read these before claiming anything in a paper.

1. One genuine camera, one room.

- All genuine webcam sessions come from a single laptop camera in one room, so genuine PCE magnitudes are inflated by shared background texture between enrollment and test sessions.
- The lowest genuine PCE (392) still shares the room's background with its enrollment session.
- Impostor numbers, by contrast, are over many cameras and generators.
- A publishable study needs **≥10 distinct webcams**, multiple subjects, lighting conditions, low light, and sessions days apart.

2. No real virtual camera was exercised.

- v4l2loopback is not installed on the test machine, and loading it needs root.
- The virtual-node classifier is verified against a fake sysfs tree that mirrors v4l2loopback's real layout, and the challenge against replayed frames.
- Before publication, run the real attacks: `sudo modprobe v4l2loopback card_label="Integrated Camera"`, then feed it with `ffmpeg -re -i fake.mp4 -f v4l2 /dev/videoN`, then OBS Virtual Camera, then DeepFaceLive.

3. Adaptive attackers.

- A user-space attacker who polls the physical camera's brightness (`VIDIOC_G_CTRL`) could modulate injected frames to match the challenge.
 - Mitigations to evaluate:
 - challenge controls whose effect is hard to emulate convincingly (exposure time → motion-blur and noise-level change; white balance → per-channel gains)
 - tighter latency bounds
 - checking that the sensor noise level rises with gain, which is physically mandated
4. **Kernel-level attacks are out of scope.** A malicious driver can fake sysfs, controls and frames. Hardware-rooted attestation (e.g. signed camera firmware, Android Key Attestation / Play Integrity) is the complement.
 5. **Gate 3 is uncalibrated, and its Haar face detector is weak.** It found faces in 0–27 of 90 frames on webcam clips where a face is plainly visible, so it often abstains. Its flags on fakes and real clips should be read from §7.5, not assumed. Replacing the detector (e.g. OpenCV YuNet) and calibrating thresholds is the first Gate 3 task.
 6. **Blind PRNU has no hard decision power.** Uploaded files can only be checked against a reference fingerprint, which a real KYC flow could obtain at device registration.
 7. **Platform.** The desktop Layer 1 engine is Linux/V4L2-only. The equivalents are Media Foundation (`IAMVideoProcAmp`) on Windows and AVFoundation on macOS. The Android SDK (Kotlin) is a separate prototype; its JSON verdict is accepted by `evaluate_client_attestation` but it does not yet stream frames.
 8. **DF40 clips are processed releases.** Face crops and re-encodes of public videos. They test "not this sensor" (reference mode) properly, but they are not a stand-in for live injected streams.

9. Path to publication

Claim to make. A sensor-bound, gated architecture detects digital injection and synthetic video in live capture without any learned model. It does so by:

- (i) attesting the physical capture device
- (ii) proving causal control of that sensor over the frame stream with a randomised challenge
- (iii) verifying the frames' PRNU against a fingerprint enrolled under the attested identity

The evidence so far: 0/100 foreign-source accepts (max PCE 16.9 vs threshold 60), 58/58 genuine accepts (min PCE 392; single camera, single room), a challenge null false-pass bound of 9×10^{-5} , and sub-second compute on a laptop CPU.

Do not claim that blind PRNU or Gate 3 detect deepfakes in general; the data here says otherwise.

Experiments still needed (in priority order):

1. Multi-device genuine study: ≥ 10 webcams (USB and built-in), ≥ 10 subjects, varied light, sessions across days, **with enrollment and test in different scenes** (removes the §7.2 leakage). Report genuine accept rate and the PCE distribution vs N frames (15/30/45/90).
2. Real injection attacks A1–A3 and A8 with v4l2loopback, OBS, DeepFaceLive and an `LD_PRELOAD` shim around `VIDIOC_DQBUF`. Report per-gate detection.
3. Adaptive-attacker study for the challenge (Section 8.3), with multi-control challenges.
4. Ablation:
 - each gate alone vs combined (coverage matrix with measured cells)
 - compute and latency per gate
 - server load saved by early termination
5. Gate 3 calibration on DF40/FF++ with a modern face detector, or replace it with a lightweight frequency model, clearly separated from the training-free core.
6. Comparison with ISO/IEC 30107-3 PAD baselines and with commercial IAD claims; position against CEN/TS 18099.

Related work to position against:

- PRNU source identification (Lukáš 2006; Chen 2008; Goljan 2009)
- PRNU in stabilised video (Taspinar 2016; Mandelli 2020)
- GAN fingerprints (Marra 2019)

- active-illumination liveness, e.g. *Face Flashing* (Tang et al., NDSS 2018), which challenges the scene with screen light rather than the sensor's ISP controls
- rPPG liveness / FakeCatcher (Ciftci et al. 2020)
- deepfake benchmarks (FaceForensics++ 2019; DF40 2024)

Do a proper literature search for prior "camera control challenge-response" work before claiming novelty for Gate 1b.

Venues:

- applied security: ACSAC, AsiaCCS, ACNS
- media forensics: IEEE WIFS, ACM IH&MMSec
- journals: IEEE TIFS, Elsevier FSI: Digital Investigation

For a patent, the claim structure in the project brief maps onto §3: interception at the HAL/V4L2 layer, the sensor challenge and PRNU binding to the attested identity, and gated termination.

10. Reproduce

```
python run_tests.py                                # 57 tests
python benchmarks/fetch_datasets.py                # ~1 GB: VISION subset + DF40 + T2V samples
python benchmarks/capture_webcam_sessions.py --sessions 4 --gap 30 --tag "<conditions>"
python benchmarks/run_benchmark.py --live /dev/video0 --live-trials 20
```

11. References

- J. Lukáš, J. Fridrich, M. Goljan. Digital camera identification from sensor pattern noise. *IEEE TIFS* 1(2), 2006.
- M. Chen, J. Fridrich, M. Goljan, J. Lukáš. Determining image origin and integrity using sensor noise. *IEEE TIFS* 3(1), 2008.
- M. Goljan, J. Fridrich, T. Filler. Large scale test of sensor fingerprint camera identification. *Proc. SPIE* 7254, 2009.
- M. K. Mihçak, I. Kozintsev, K. Ramchandran. Spatially adaptive statistical modeling of wavelet image coefficients and its application to denoising. *ICASSP* 1999.
- S. Taspinar, M. Mohanty, N. Memon. Source camera attribution using stabilized video. *IEEE WIFS* 2016.
- S. Mandelli, P. Bestagini, L. Verdoliva, S. Tubaro. Facing device attribution problem for stabilized video sequences. *IEEE TIFS* 15, 2020.
- F. Marra, D. Gagnaniello, L. Verdoliva, G. Poggi. Do GANs leave artificial fingerprints? *IEEE MIPR* 2019.
- D. Shullani, M. Fontani, M. Iuliani, O. Al Shaya, A. Piva. VISION: a video and image dataset for source identification. *EURASIP J. on Information Security*, 2017.
- Z. Yan et al. DF40: Toward next-generation deepfake detection. *NeurIPS Datasets & Benchmarks* 2024.
- A. Rössler et al. FaceForensics++: Learning to detect manipulated facial images. *ICCV* 2019.
- D. Tang et al. Face Flashing: a secure liveness detection protocol based on light reflections. *NDSS* 2018.
- U. A. Ciftci, I. Demir, L. Yin. FakeCatcher: Detection of synthetic portrait videos using biological signals. *IEEE TPAMI*, 2020.
- G. Farnebäck. Two-frame motion estimation based on polynomial expansion. *SCIA* 2003.
- ISO/IEC 30107-3:2023 Biometric presentation attack detection, Part 3. CEN/TS 18099:2024 Biometric data injection attack detection.